Suggested Claude Code Agents & Skills

The Golden Fur Capstone Repo & Second-Brain Vault Repo

Based on a read-through of the GoldenFur-Proposal-Revisions document (system scope, modules, and adviser feedback) and the AI Second Brain conversation summary (vault/agent/skill architecture), below are the recommended .claude/agents and .claude/skills for each repo, ranked from most to least required. Ranking reflects business/technical risk, frequency of use, and how foundational each item is to everything built after it.

Important: everything below is developer tooling. Agents and skills run only inside Claude Code, on your machine, while you're writing code — they are never deployed, never run on the live website, and are not reachable by Mike Ocsan, his staff, or customers. Their output is ordinary source code that you commit and deploy as usual; once that code is live, Claude Code is no longer in the loop.

Repo 1 — The Golden Fur (capstone-repo)

Agents (.claude/agents/) — dev-time subagents

Rank	Name	What it does	Why it ranks here
1	**booking-capacity-agent**	A Claude Code subagent you delegate to while coding: scaffolds, reviews, and debugs the cage-capacity (S/M/L/XL) and overbooking-prevention logic for Grooming/Hotel/Daycare/Vet before you commit it.	This is the hardest business logic to get right and the most cited pain point (overbooking). Worth a dedicated dev-time reviewer before this code ever reaches a PR.
2	**payment-billing-agent**	A subagent for implementing and testing PayMongo (GCash/Maya) webhook handling, manual reconciliation logic, and the Credit Balance ledger against sandbox data — used locally, never touches live transactions itself.	Payment code is the highest-stakes thing you'll write; having a subagent dedicated to catching webhook/idempotency bugs during development lowers the risk of shipping a money bug.
3	**auth-access-agent**	A subagent that helps you write and review RBAC, TOTP MFA, and Google/Facebook OAuth account-merge code as you build it, with read-mostly tool access so it can't accidentally touch production config.	Security code is easy to get subtly wrong and expensive to retrofit; a second pass from a dedicated subagent during development catches issues before they ship.
4	**report-generator-agent**	Helps you build and unit-test the Daily Sales Report and grooming-stats generation code (per branch, per payment method, coat/weight tiers) during development.	Complex aggregation logic that's easy to get subtly wrong; useful to have a subagent double-check the math and edge cases before you wire it into the app.
5	**notification-agent**	Assists in writing and testing the code for the 5 transactional email triggers and push notifications — drafting templates, trigger conditions, and failure-handling logic in your dev environment.	Cross-cuts several modules; a subagent keeps the trigger logic consistent as you touch booking, payment, and account code in different sessions.
6	**discount-compliance-agent**	Helps you scaffold the (currently inactive) Discount Module code, including senior/PWD statutory	Lower urgency since the feature is inactive — useful to have on hand for when this module actually gets built out.

		discount handling, ready for when the client turns it on.	
7	**qa-iso25010-agent**	A subagent for writing test cases and drafting the evaluation questionnaire mapped to ISO/IEC 25010:2011, used only within your dev/research workflow, not part of the shipped app.	Needed for the capstone's evaluation chapter, but only becomes useful once modules are functionally complete enough to test.
8	**db-schema-agent**	Helps you design and write migration scripts for the pets/clients/bookings/credits schema and multi-branch data isolation, run locally against your dev database.	Mostly a one-time setup task early in development rather than a recurring need — ranked last as 'do once, then stable'.

Skills (.claude/skills/) — dev-time reference material

Rank	Name	What it does	Why it ranks here
1	**paymongo-webhook-handling**	A written reference Claude Code loads while you're implementing PayMongo integration: webhook signature verification, idempotent status updates, and how to test against the GCash/Maya sandbox.	Payment logic must be correct on the first real deploy; a skill prevents you (and Claude) from re-deriving webhook security pitfalls from scratch each coding session.
2	**capacity-based-scheduling**	Documents the S/M/L/XL cage-capacity algorithm and overbooking rules so Claude Code applies them the same way anywhere in the codebase this logic gets touched.	The most complex business rule in the app; keeping it in a skill stops the logic from drifting across different files or coding sessions.
3	**rbac-totp-setup**	Reference steps for scaffolding TOTP enrollment and role-gated middleware for each of the six roles, so new protected routes/pages follow the same pattern.	Reused constantly as you add new endpoints; keeps access-control code consistent instead of hand-rolled per feature.
4	**credit-balance-ledger**	Documents the rules for converting a cancelled deposit into non-transferable, per-branch, no-expiry credit, for Claude Code to follow when writing that logic.	Subtle rules (branch-locking, no-expiry) that are easy to implement wrong without a written reference to check against.
5	**daily-sales-report-format**	Captures the exact DSR layout (per-branch, per-payment-method, separate BPI/BDO/Grabmart/Pickaroo lines) so the report-generation code matches the client's current manual format.	A precise spec Claude Code can build against, so the generated report doesn't need repeated correction against the client's real paperwork.
6	**email-notification-templates**	Templates and trigger conditions for the 5 transactional emails plus reschedule notifications, for Claude Code to reuse when writing notification code.	Repetitive but well-defined; keeps wording and trigger logic consistent as different modules are built out over time.
7	**discount-senior-pwd-compliance**	A checklist referencing Philippine senior citizen (RA 9994) and PWD (RA 10754) discount law, for when	Only needed once that module moves from inactive to in-development — good to have ready, not urgent yet.

		the discount engine code actually gets written.	
8	**iso25010-evaluation-instrument**	A template for drafting the Likert-scale questionnaire mapped to ISO/IEC 25010's 8 quality characteristics, used for the capstone's written evaluation deliverable.	Supports the research/documentation side of the capstone rather than the app itself — most useful near defense time.

Repo 2 — Second-Brain Vault (second-brain-vault)

Agents (.claude/agents/)

Rank	Name	What it does	Why it ranks here
1	**vault-librarian**	Turns unstructured input (call notes, clippings, voice-to-text) into a properly filed, frontmatter-tagged Obsidian note. Read/Write/Grep/Glob only.	This is the entire point of the vault — without a reliable filing agent, notes never get captured in the first place.
2	**weekly-reviewer**	Reads notes modified in the last 7 days and writes a rollup summary into Areas/Reviews/, read-only except for its own output file.	Prevents the vault from becoming a write-only dump; is what actually makes the notes useful to look back on.
3	**backlink-curator**	Scans new/edited notes and inserts [[wikilinks]] to related existing notes, flags orphaned notes with no links.	A second-brain's value compounds through connections between notes, not isolated entries; this keeps the graph from decaying.
4	**research-capture-agent**	Files literature/interview material (e.g. GoldenFur capstone sources) into Resources/ with proper citation metadata.	Useful cross-repo bridge (capstone research -> vault) but secondary to getting basic filing and review working first.
5	**skill-agent-auditor**	Reviews any third-party skill or agent file for prompt-injection risk before it's added to the vault or repo.	Only matters once you start importing skills/agents from outside sources — a safety net, not a daily-use agent.

Skills (.claude/skills/)

Rank	Name	What it does	Why it ranks here
1	**note-filing**	How to file a raw note: which subfolder (Inbox/Projects/Areas/Resources/Archive), required frontmatter, naming convention.	The load-bearing skill every filing action depends on; nothing else in the vault works well without this being solid first.
2	**frontmatter-schema**	Canonical YAML frontmatter fields (type, tags, date, source, status) enforced across all notes.	Directly determines whether note-filing and weekly-reviewer can reliably query/filter notes later.
3	**cross-linking**	Rules for when and how to add [[wikilinks]] between related notes during filing or review.	Powers backlink-curator; ranked after the schema it depends on.
4	**weekly-review-format**	Template/structure for the rollup weekly-reviewer writes into Areas/Reviews/.	Narrow in scope — only needed to standardize one agent's output.

5	**agents-md-maintenance**	Keeps AGENTS.md canonical and symlinked to CLAUDE.md/GEMINI.md so the vault stays portable across tools.	A one-time/occasional housekeeping skill rather than something invoked on every note.
6	**skill-security-audit**	Checklist for vetting any publicly-shared skill or agent (per the 2026 prompt-injection audit findings) before adoption.	Lowest frequency of use — only triggered when importing something you didn't write yourself.

Note
Rankings assume typical build order: get the risky/foundational pieces (booking, payments, filing) solid first, then layer reporting, compliance, and evaluation/housekeeping work on top. Re-rank freely once actual sprint priorities are set.

For GoldenFur, the vault's note-filing agents are a genuine second product surface (a personal knowledge tool you run yourself), whereas everything in the GoldenFur section exists purely to help produce the deployed system's source code faster and more correctly — it has no runtime presence in that system.